IMEN891K / IDSC501 / AXGS-501 · DATA ENGINEERING · FALL 2026

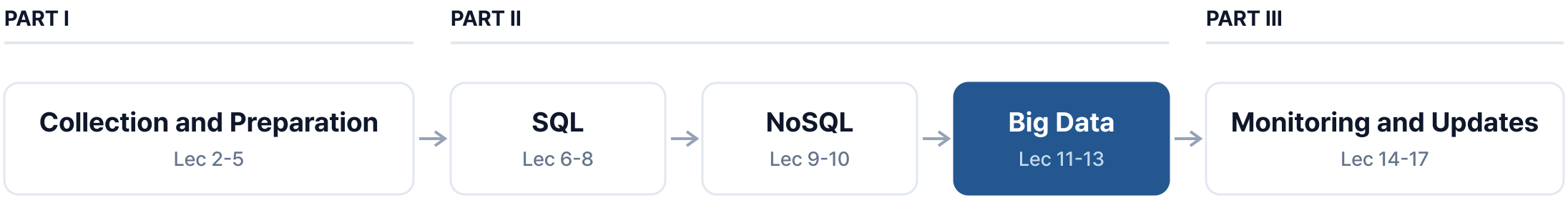
11.

Big Data I: Batch Processing

Sangwoo Mo · POSTECH

Industrial & Management Engineering

Where We Are


PART I
PART II
PART III
Collection and Preparation
Lec 2-5
SQL
Lec 6-8
NoSQL
Lec 9-10
Big Data
Lec 11-13
Monitoring and Updates
Lec 14-17

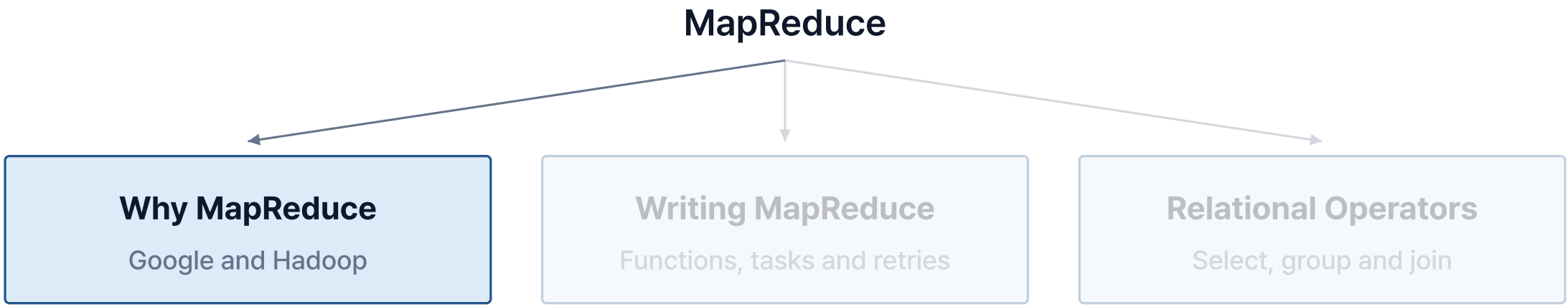
Outline: Big Data I

01	**MapReduce**	write map and reduce functions for distributed jobs
02	**Spark**	compose RDD operations, then describe table queries with DataFrames

01 · MapReduce

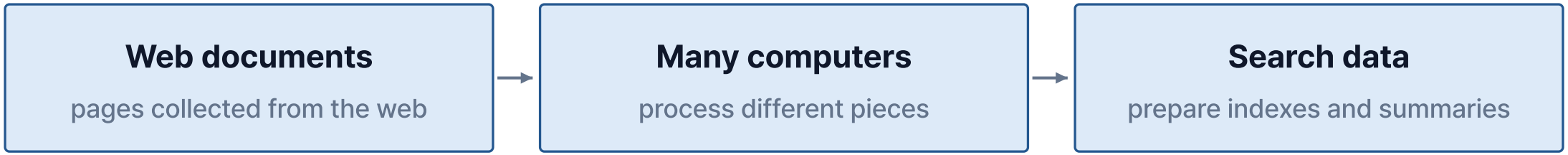
01	**MapReduce**	write map and reduce functions for distributed jobs
02	**Spark**	compose RDD operations, then describe table queries with DataFrames

§ Why MapReduce



Google Had a Data Problem

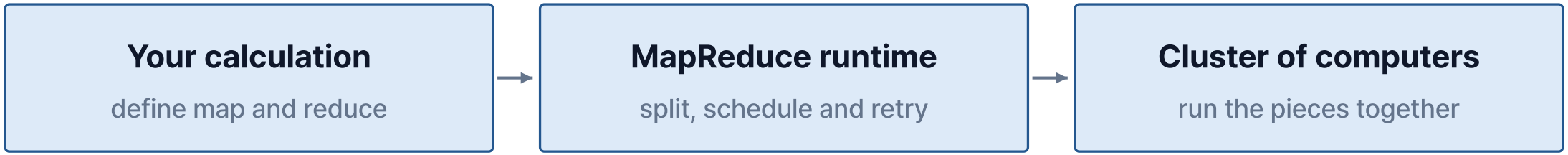
Google needed to turn large collections of web pages and request logs into useful data



Simple calculations became difficult when the input needed many computers

Google Publishes MapReduce

In 2004, Jeffrey Dean and Sanjay Ghemawat at Google published the MapReduce paper



Programmers describe the calculation; the runtime coordinates the computers

Hadoop Makes the Idea Available

Doug Cutting and Mike Cafarella developed Hadoop from the Nutch search project



Hadoop brought distributed storage and MapReduce to an open-source community

The Name Hadoop

The elephant in the logo has a family story behind it



- Hadoop was the name of Doug Cutting's son's toy elephant
- The family nickname became the software project's name

Why Many Machines

Distributed processing divides work across computers when one computer is insufficient

One computer reads everything

All input rows

one computer does all the work

Three computers share the reading

Computer 1: one piece

Computer 2: one piece

Computer 3: one piece

Same question, same answer; different computers handle different rows

Different computers read different rows to answer the same question

Scale Up or Scale Out

Scale up adds resources to one machine; scale out adds machines

Scale up

One larger machine

more memory / processing power

Simpler, until one machine is not enough

Scale out

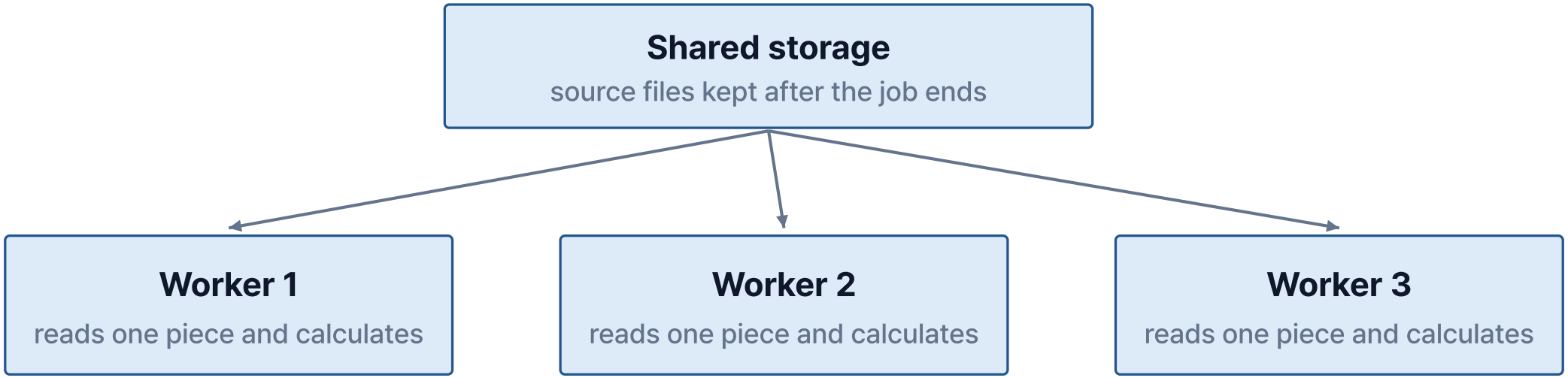
Worker 1 **Worker 2** **Worker 3**

More capacity, plus coordination and transfers

Extra machines add capacity and coordination work

Storage and Compute

Storage keeps the files; workers are processes that read files and calculate results

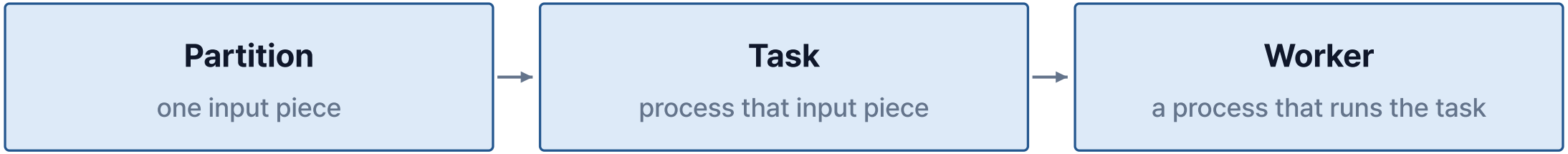


Storage keeps source data; workers run the computation

Durable input and temporary computation have different jobs

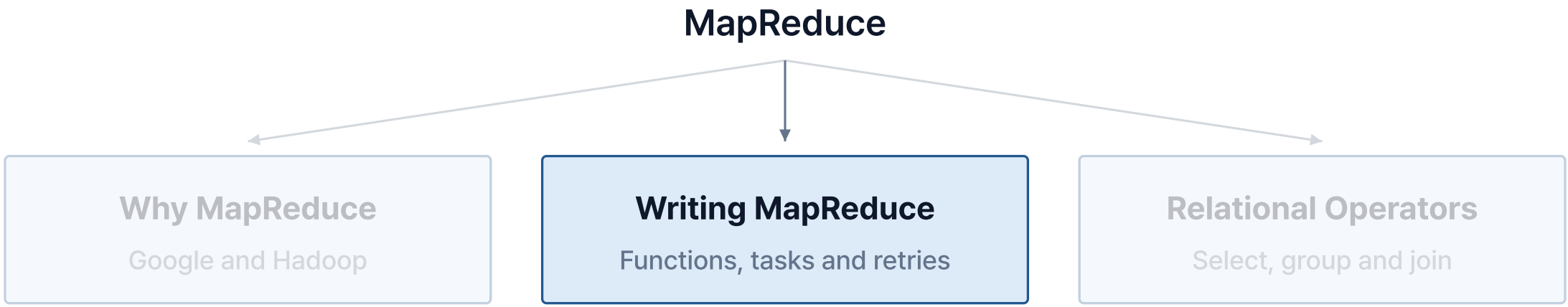
Distributed Files and Tasks

HDFS stores file blocks; MapReduce assigns computation to workers



- A distributed file system divides a large file into blocks
- Replicas keep another copy if a storage server fails
- Workers run tasks over pieces of the stored input

§ Writing MapReduce



Word Count

The input is a collection of documents; the output is one count per word

Document ID	Document contents
d1	spark makes data simple
d2	spark processes data

Question: How many times does each word occur across all documents?

Map: Emit a Word and One

Map turns one document into several (word, 1) pairs

```
map(document_id, text):
    for word in text.split():
        emit(word, 1)
```

Input text	Emitted pairs
spark processes data	(spark,1), (processes,1), (data,1)

Shuffle and Reduce

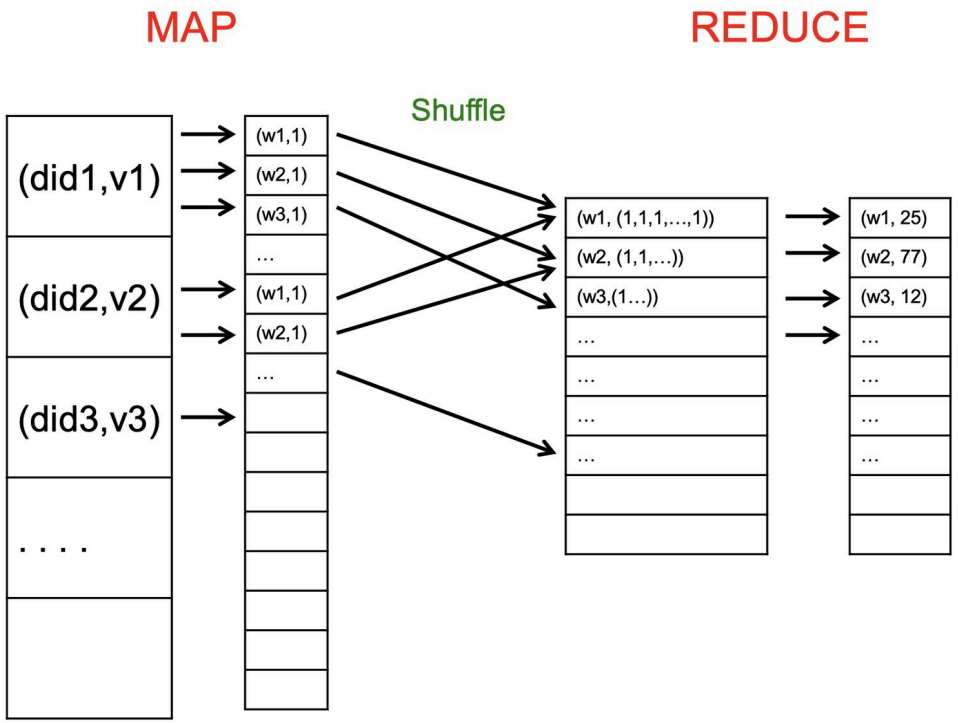
The system groups identical words; reduce adds their counts

Key after shuffle	Values	Reduce result
spark	[1, 1]	(spark, 2)
data	[1, 1]	(data, 2)
processes	[1]	(processes, 1)

```
reduce(word, counts):
    emit(word, sum(counts))
```

Word Count: The Full Data Flow

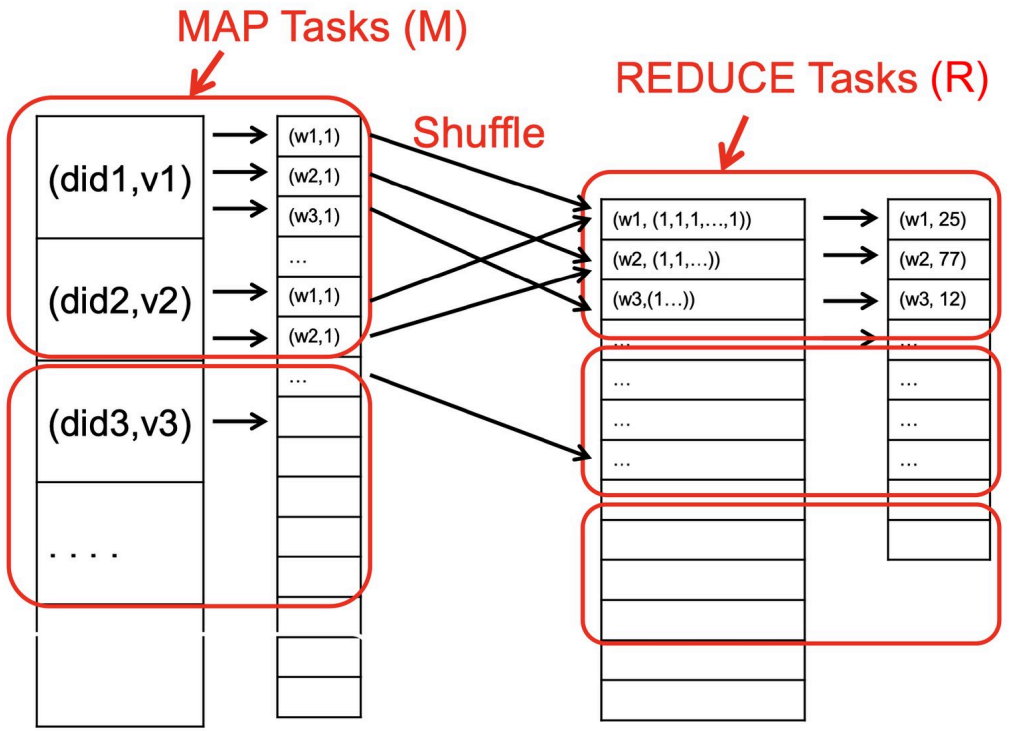
did identifies a document, v is its text, and w is a word



Map emits (word, 1); shuffle groups words; reduce adds each group

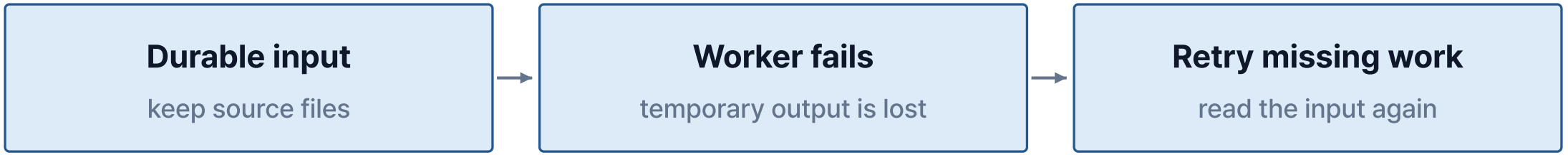
Map Tasks and Reduce Tasks

A task is one unit of work; a worker process can run multiple tasks over time



Map tasks split the input; reduce tasks split the grouped keys

A Worker Can Fail



Retry needs available source data and a repeatable computation

The Job Also Needs Setup

A Hadoop MapReduce job connects your functions, data types and file paths

```
job.setMapperClass(WordMapper)
job.setReducerClass(WordReducer)
job.setMapOutputKeyClass(Text)
job.setMapOutputValueClass(IntWritable)
setInputPath(job, input_path)
setOutputPath(job, output_path)
job.waitForCompletion(True)
```

Demo: Follow the Grouped Count

Sensor example: count readings at or above 80°C by machine

Back | Step | Reset

Input partitions	Map: keep + emit	Shuffle: group keys	Reduce: add
M1 70 | M2 90 | M1 85			
M1 95 | M3 75 | M2 60			
M3 88 | M1 82 | M2 78			

Input only: predict which readings pass temp >= 80 and how many remain.

Track M1 from three input partitions to one result

Answer: Follow the Grouped Count

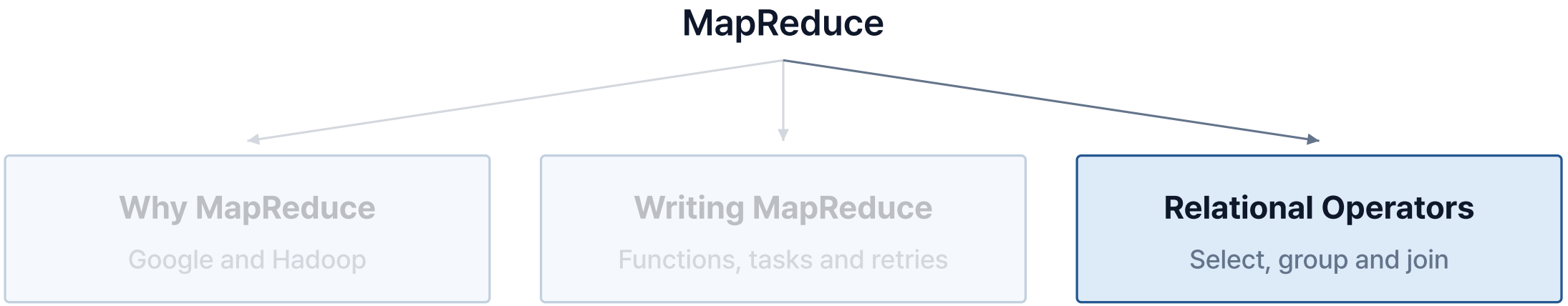
Map emits five pairs, shuffle forms three keys, reduce returns three counts



Selection: temp >= 80 | Key: machine | Value: 1

The output counts sum to five retained readings

§ Relational Operators



Relational Operations in MapReduce

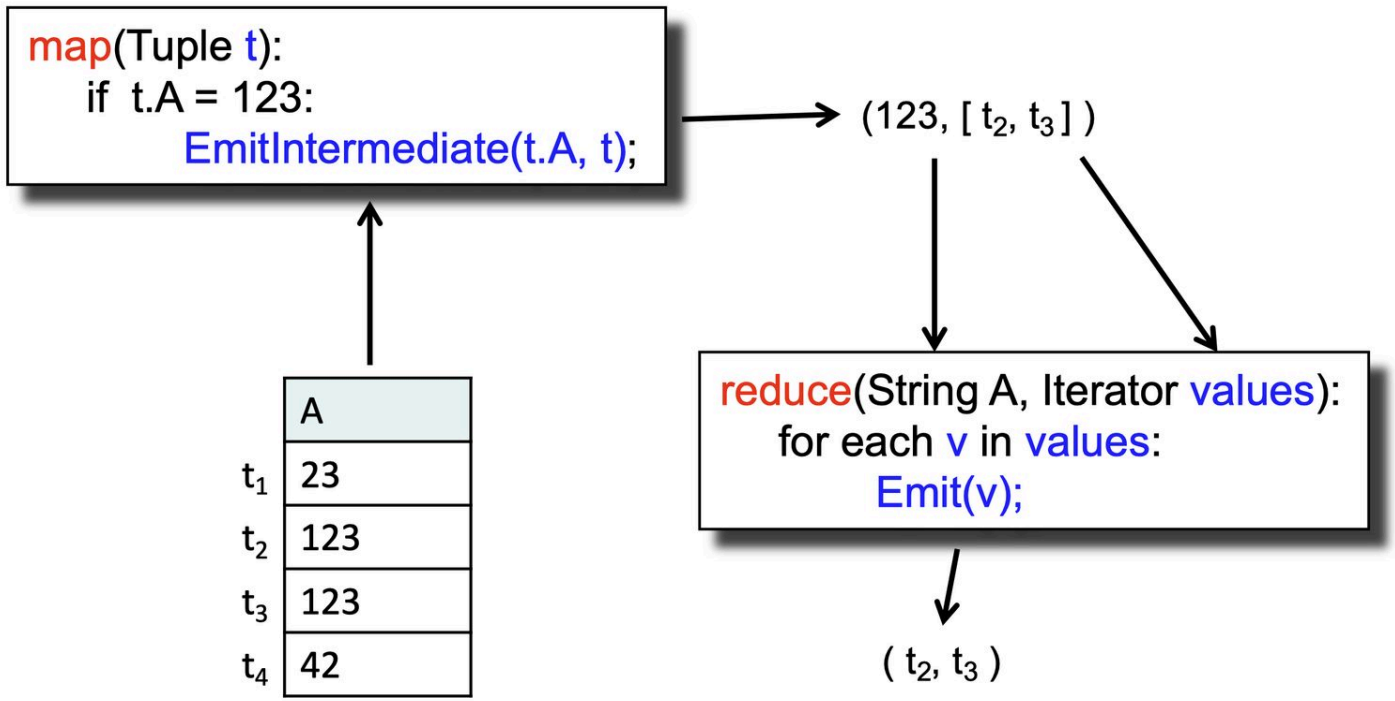
SQL names the result; MapReduce requires you to write how records produce it

SQL operation	What the MapReduce program must decide
WHERE	which rows map emits
GROUP BY + SUM	grouping key and values to add
JOIN	join key, source tags and matching pairs

The same familiar SQL operation requires explicit record-processing code

Selection in MapReduce

SELECT * FROM R WHERE A = 123 keeps only the two matching rows



The mapper checks the condition and emits the retained rows

A Map-Only Selection

A filter can finish in map; Hadoop supports map-only jobs by setting reducers to zero

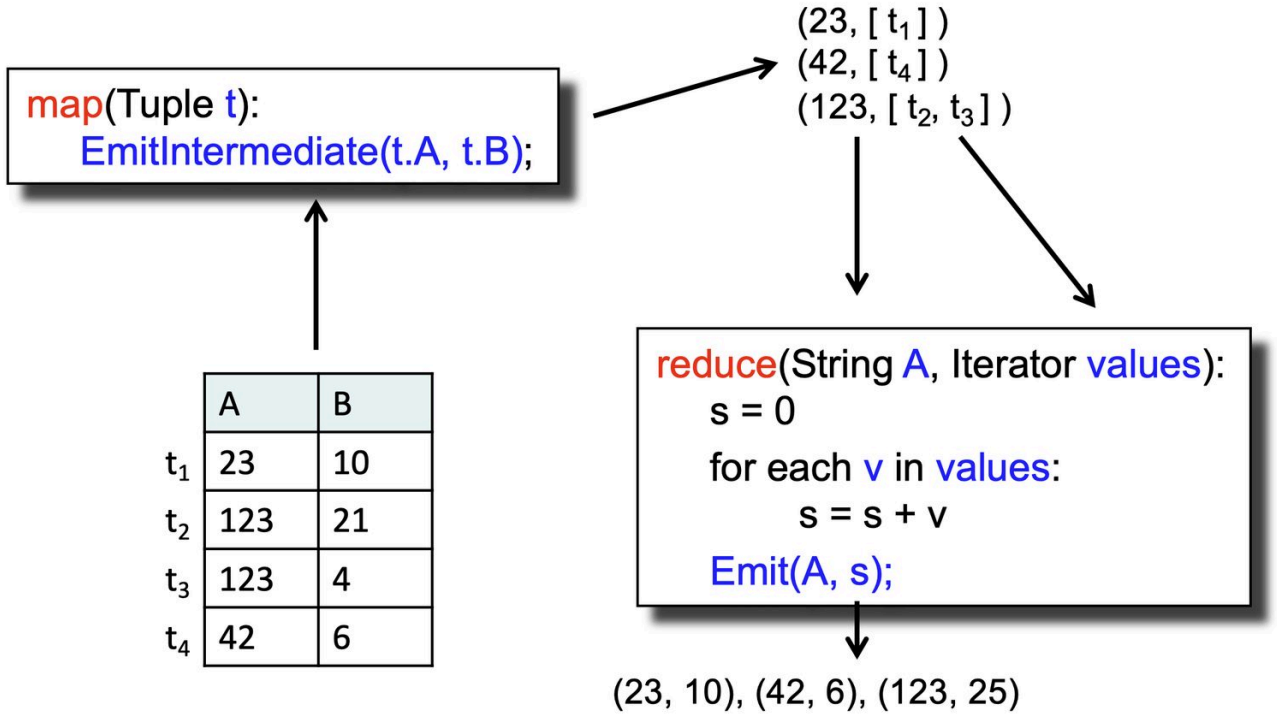
```
map(Tuple t):
    if  t.A = 123:
            EmitIntermediate(t.A, t);
```

```
reduce(String A, Iterator values):
    for each v in values:
        Emit(v);
```

Only rows passing A = 123 need to be emitted

Grouping in MapReduce

SELECT A, SUM(B) FROM R GROUP BY A adds B values within each A

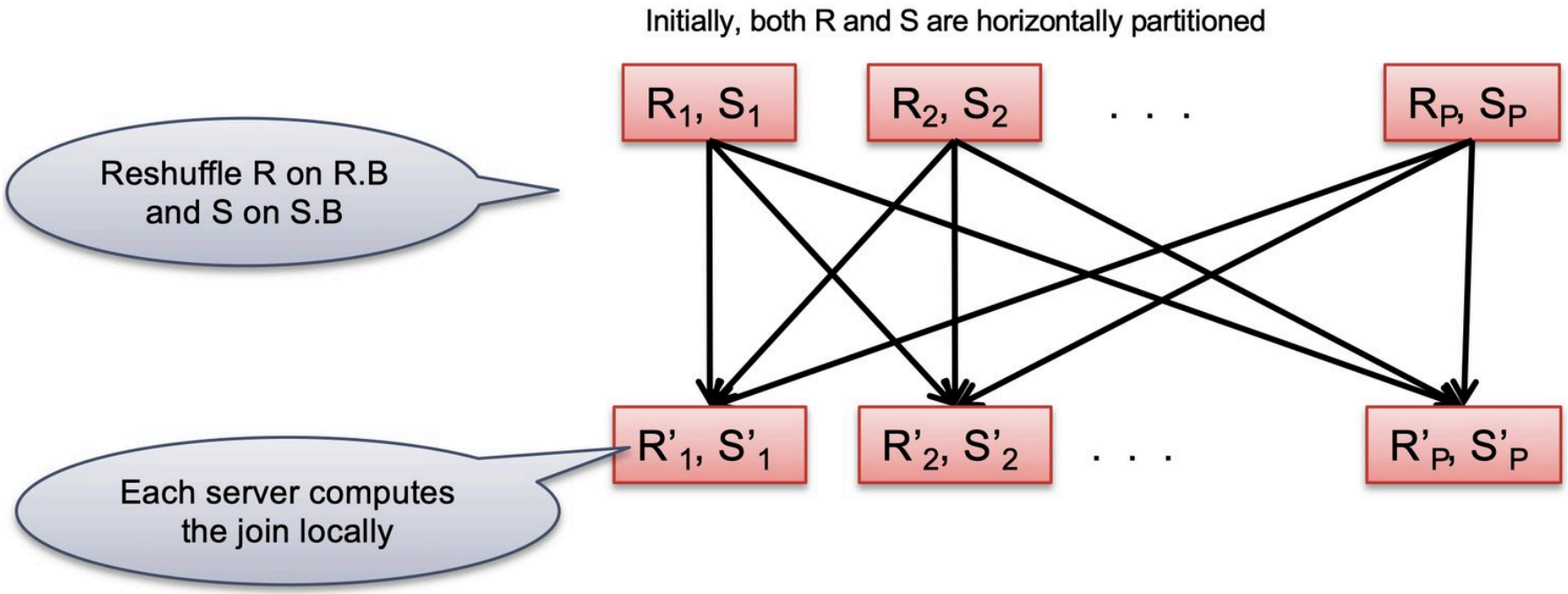


	A	B
t_1	23	10
t_2	123	21
t_3	123	4
t_4	42	6

Key 123 groups values 21 and 4; reduce returns 25

Shuffle Join: Move Both Inputs

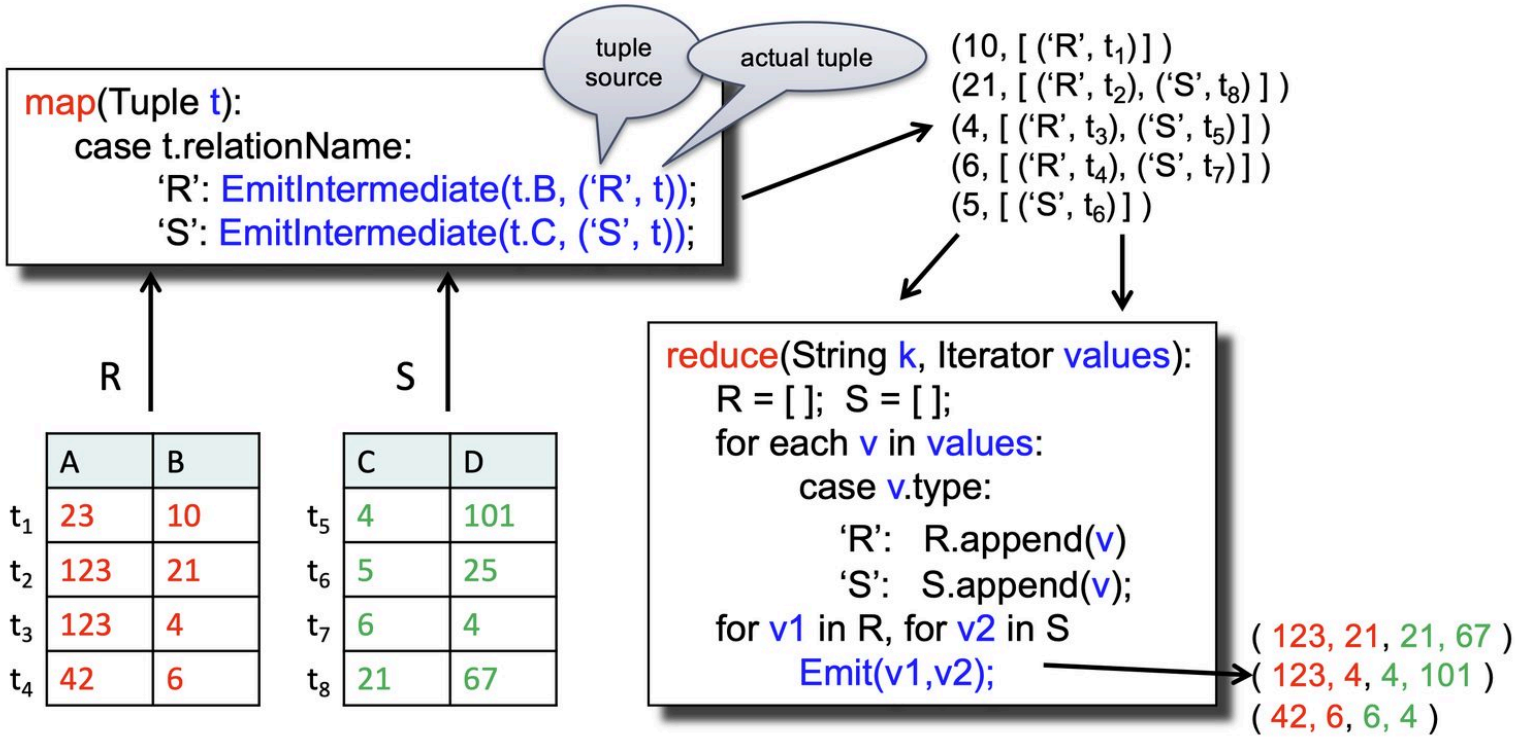
Matching join keys must meet even when the two input tables start on different servers



Redistribute both inputs by the join key, then join locally

Shuffle Join: Match Rows Within a Key

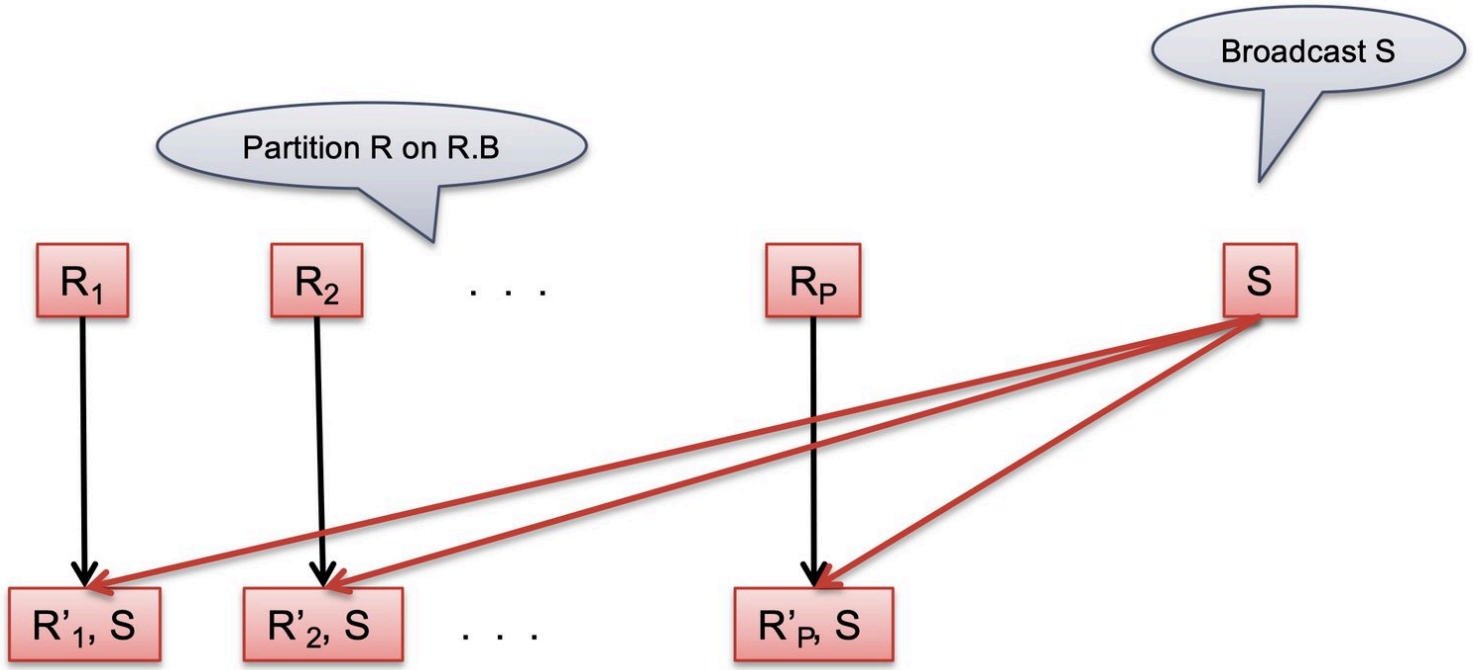
A source tag tells reduce which rows came from R and which came from S



Within one key, reduce pairs each R row with each matching S row

Broadcast Join: Copy the Small Input

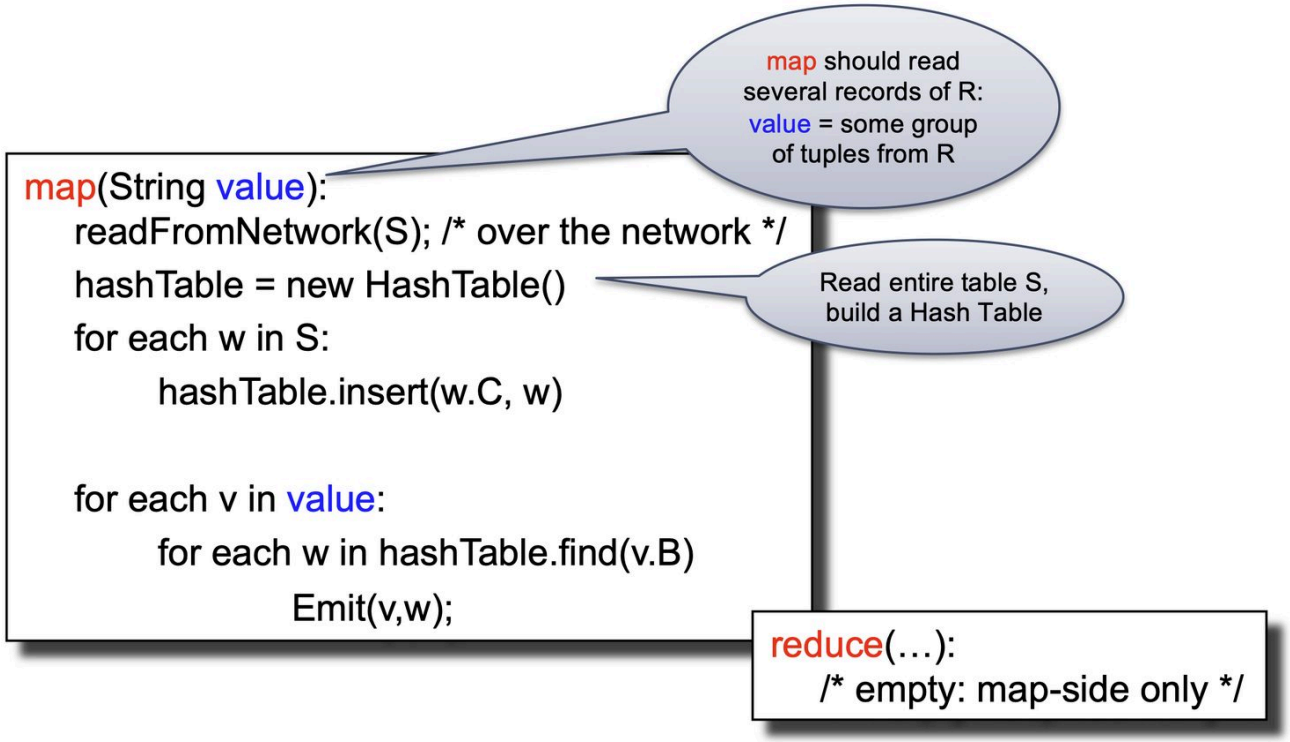
If S is small enough, each worker can receive a copy while R stays partitioned



Copy S to the workers instead of shuffling the large input

Broadcast Join: Work Inside Map

Each mapper builds a lookup table for S and probes it with its own R rows



The join can finish in map when each worker holds the small input

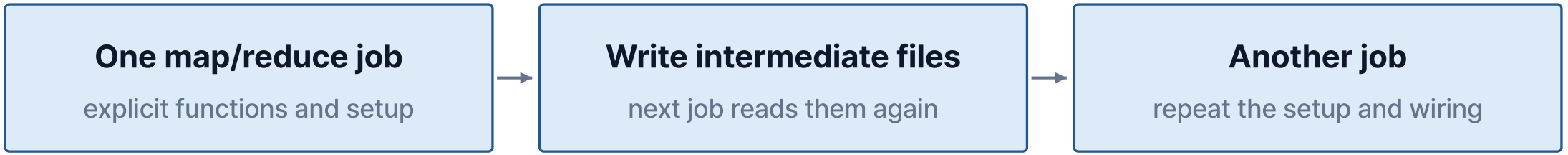
Two Join Strategies

The data sizes help determine which input movement is practical

	Shuffle join	Broadcast join
Input movement	Redistribute both inputs by key	Copy the small input to each worker
Useful when	Both inputs are large	One input fits in each worker’s memory
Main cost	Network transfer and uneven keys	Copies and memory for the small input

Why Move Beyond MapReduce

More complex calculations require more job code and intermediate data handling

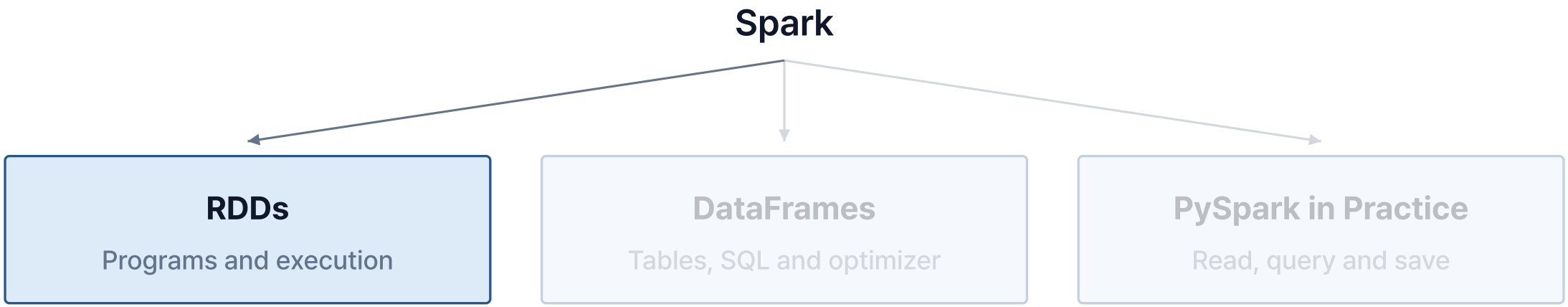


- Complex queries need several connected jobs
- You write the steps and the intermediate data flow

02 · Spark

01	MapReduce	write map and reduce functions for distributed jobs
02	**Spark**	compose RDD operations, then describe table queries with DataFrames

§ RDDs


Spark
RDDs
Programs and execution
DataFrames
Tables, SQL and optimizer
PySpark in Practice
Read, query and save

Spark Begins at Berkeley

A university research project grew into an open-source processing engine



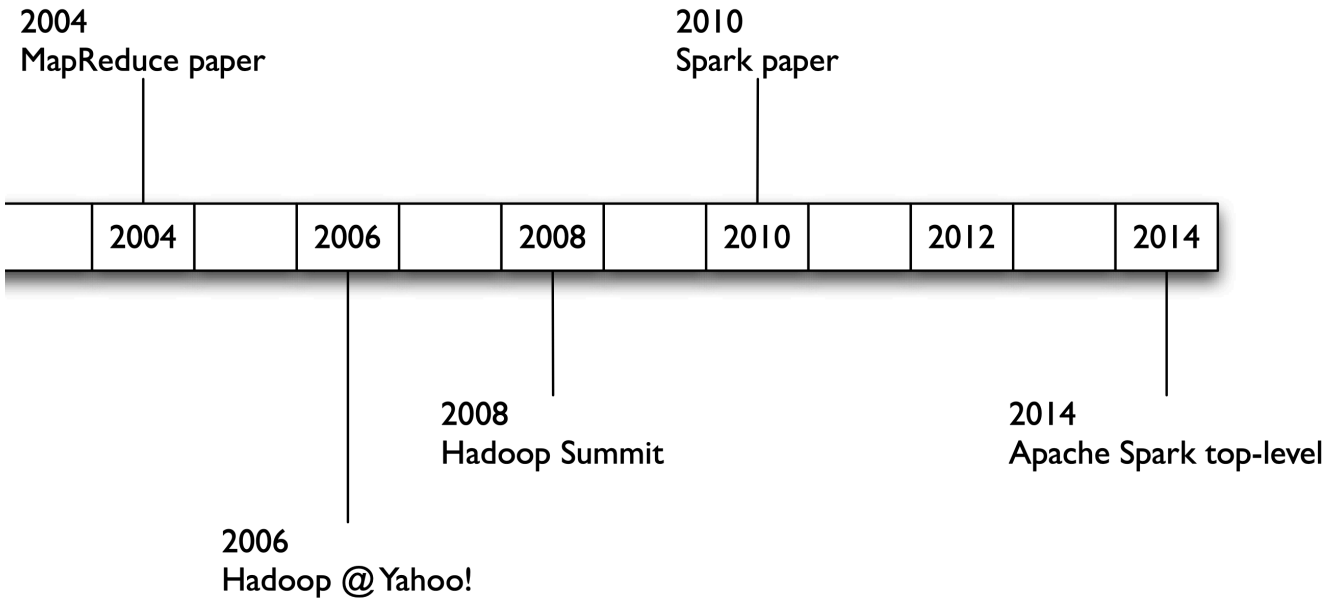




- 2009: research at UC Berkeley
- 2010: open-source release and the Spark paper
- 2013: moved to Apache; 2014: became a top-level Apache project

Spark: A Brief History

The MapReduce paper appeared in 2004; the Spark paper followed in 2010



A research idea became an open-source project and a broader community

Start a Spark Session

PySpark is the Python interface to Spark; spark is our entry point for reading data

```
from pyspark.sql import SparkSession

# Create or reuse the Spark entry point named spark
spark = (SparkSession.builder
    .appName("hot-readings")
    .getOrCreate())
```

A Shorter Word Count Program

The same word-count task needs much less job wiring in the Spark example



WordCount in 50+ lines of Java MR

```
val f = sc.textFile(inputPath)
val w = f.flatMap(l => l.split(" ")).map(word => (word, 1)).cache()
w.reduceByKey(_ + _).saveAsText(outputPath)
```

WordCount in 3 lines of Spark

Compose operations instead of wiring a new job around each pair of functions

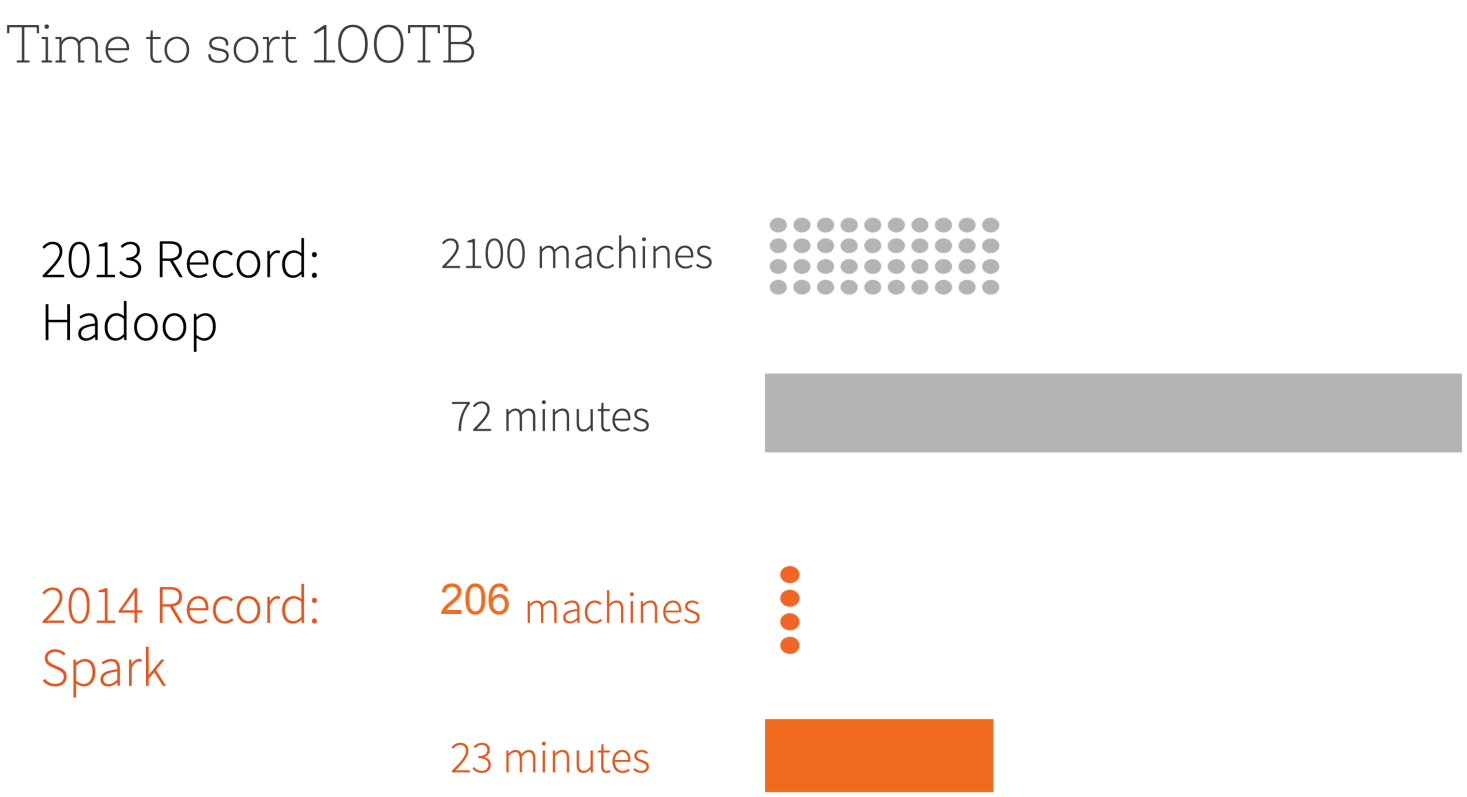
Word Count with Spark RDDs

Read lines, split words, count by key and save through a chain of operations

```
sc = spark.sparkContext
lines = sc.textFile("documents/")
words = lines.flatMap(lambda line: line.split())
counts = (words.map(lambda word: (word, 1))
    .reduceByKey(lambda a, b: a + b))
counts.saveAsTextFile("output/word_counts/")
```

Spark's 2014 Sorting Record

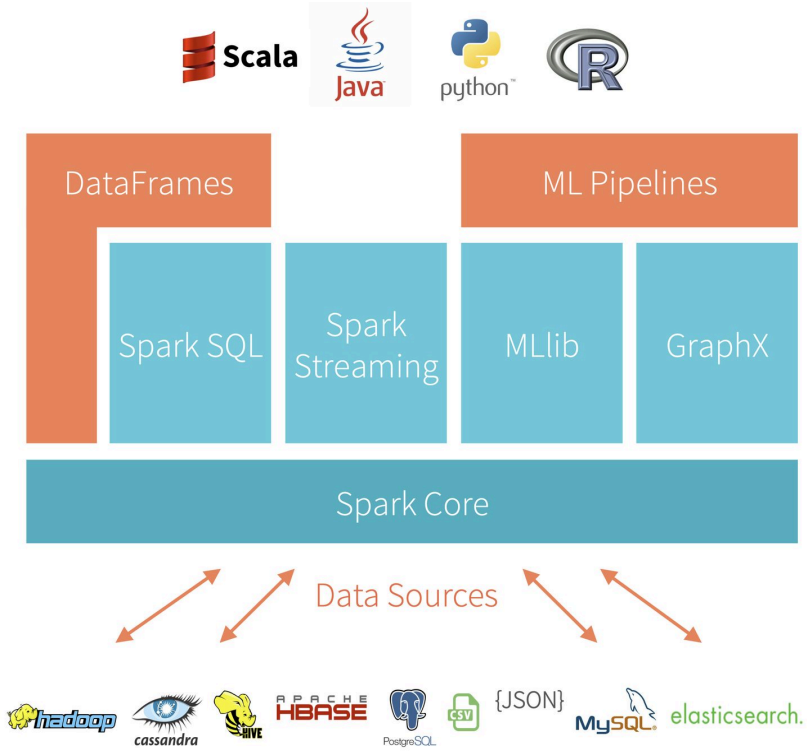
Historical sorting records drew attention to Spark as a practical processing engine



Different clusters and runs: 72 minutes in 2013, 23 minutes in 2014

Spark Libraries and Languages

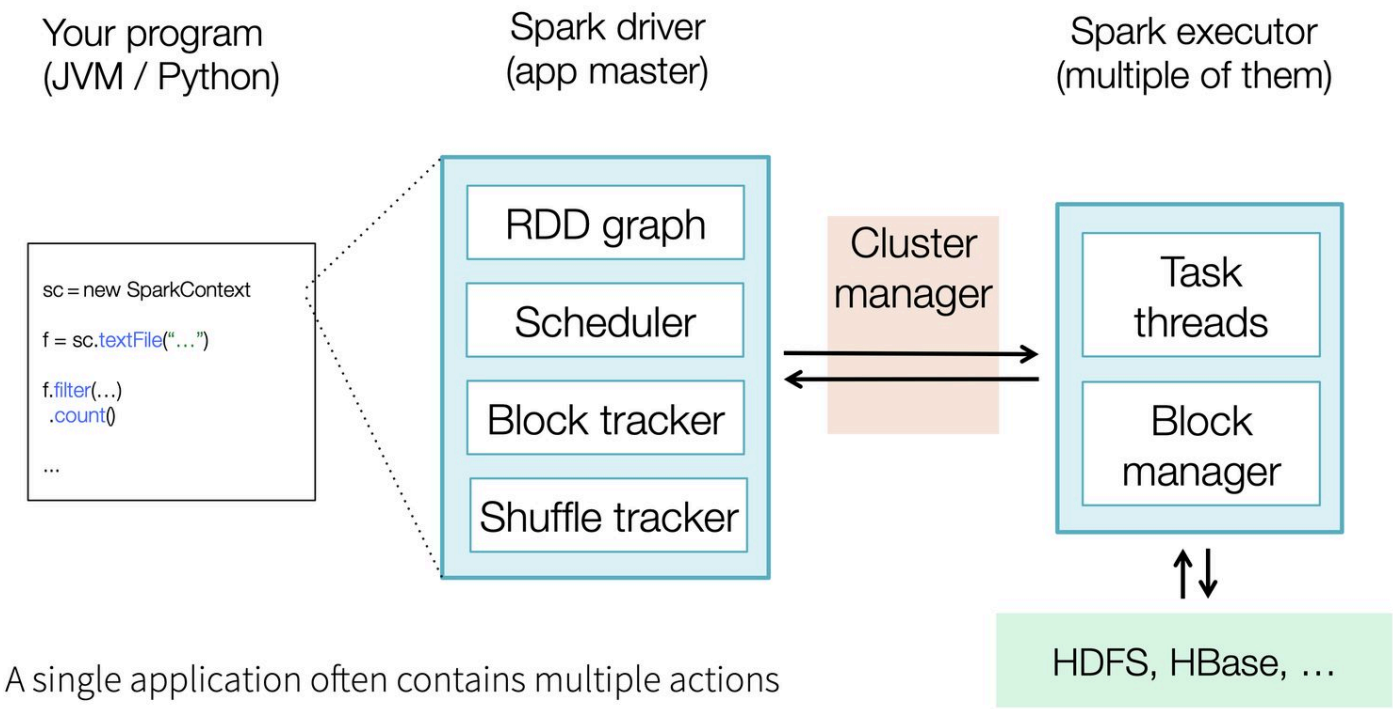
One execution engine supports several kinds of work and several programming languages



Today’s route uses Python, DataFrames and Spark SQL

Driver, Executors and Storage

The driver coordinates the program; executor processes run tasks on the data

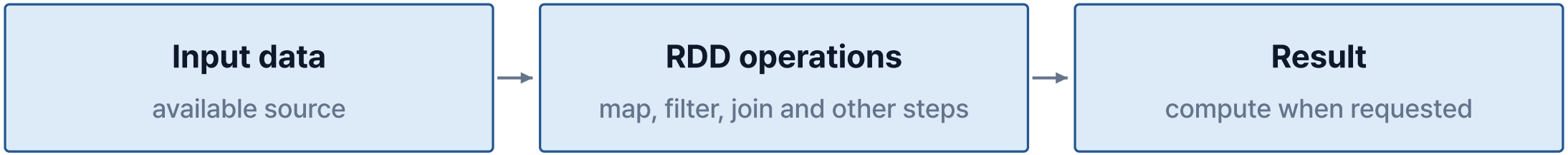


Spark handles the execution machinery behind the shorter program

Resilient Distributed Datasets

An RDD gives distributed data a programmable collection interface

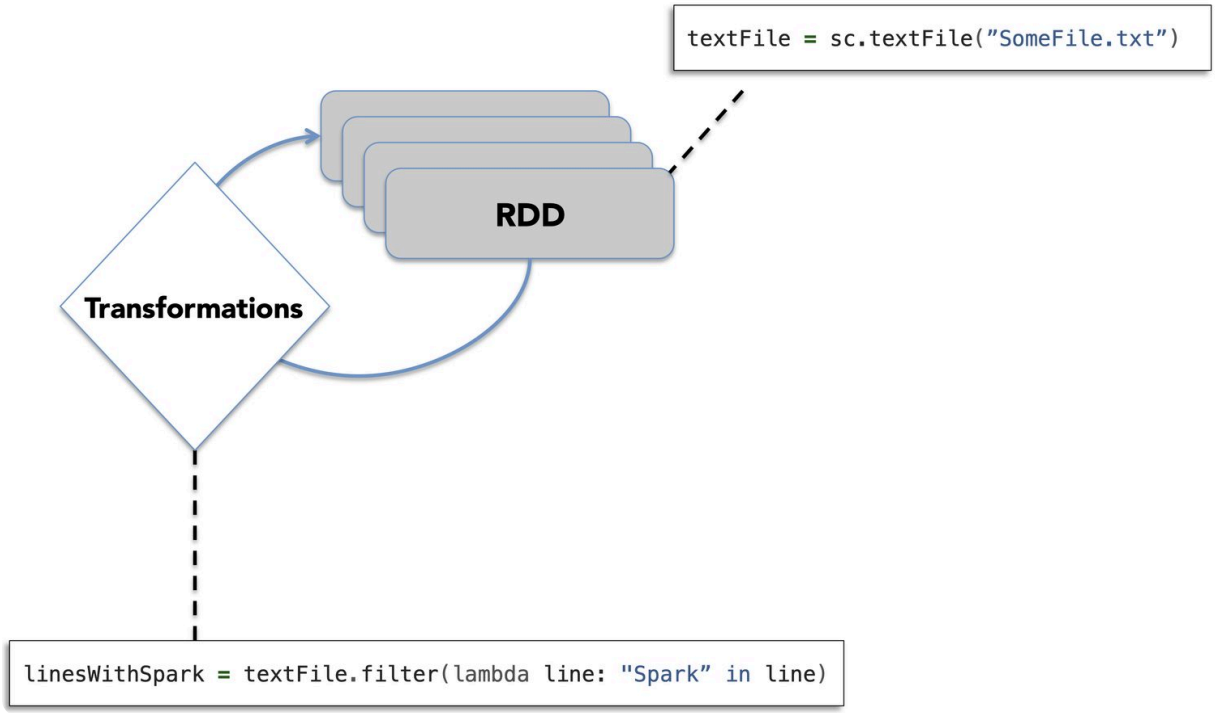
RDD: an immutable collection divided into partitions, with a record of how to rebuild it



- Distributed: partitions can run on different workers
- Resilient: lineage records how lost partitions can be recomputed

Transformations Build Another RDD

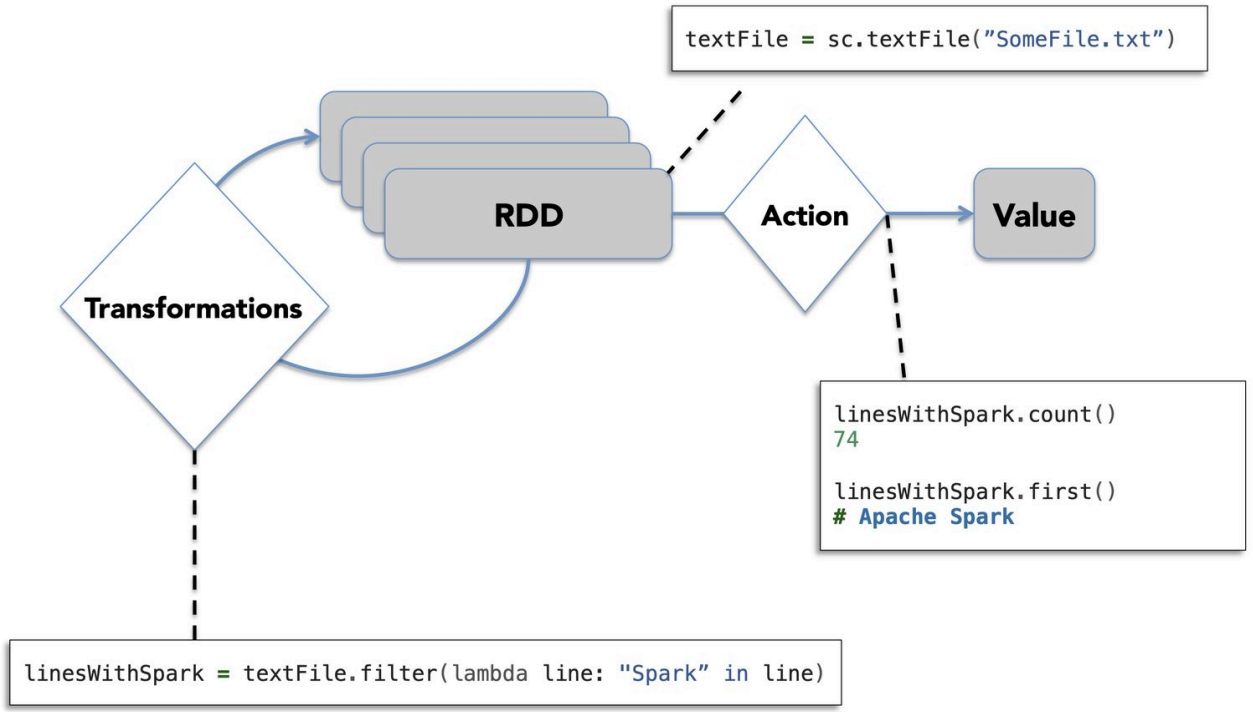
A transformation such as filter describes a new distributed collection



The program builds the calculation before requesting its result

Actions Request Results

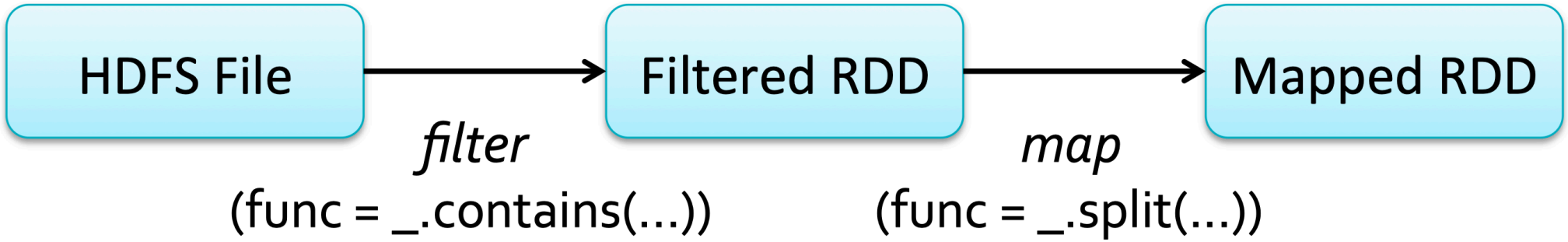
An action such as count() requests a value and triggers the required computation



Transformations return RDDs; actions return values or save output

Lineage: The Path to an RDD

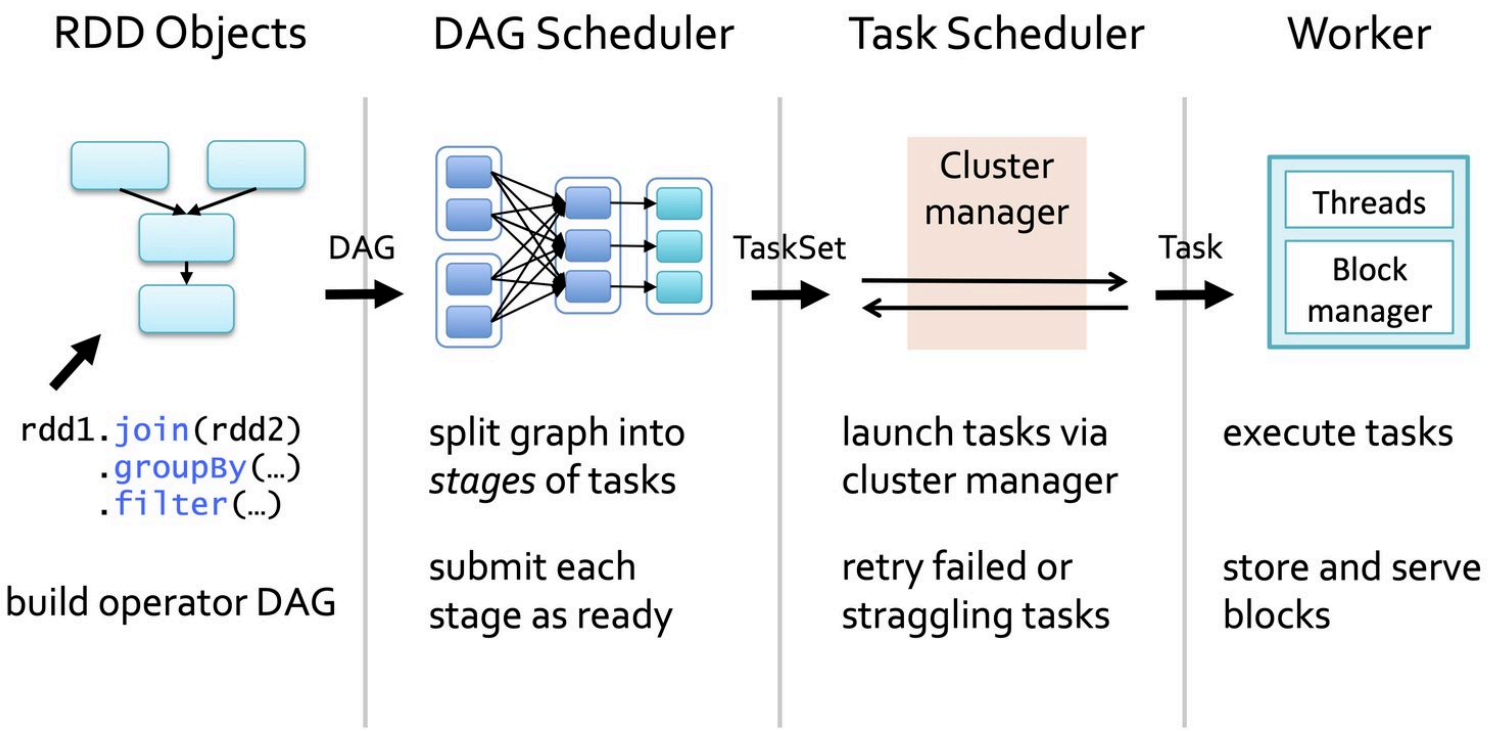
Lineage records which earlier data and operations produced the current collection



A lost partition can be rebuilt from available input and its recorded operations

RDD Execution Process

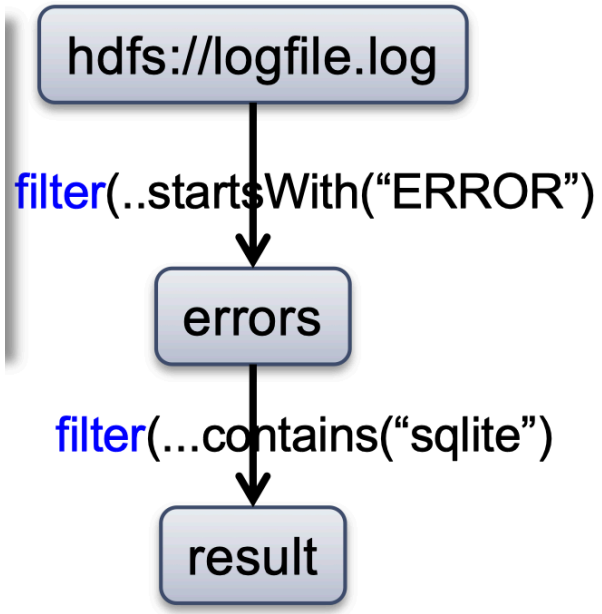
Spark turns the operation graph into stages and tasks, then assigns them to executors



A short program still runs through a coordinated distributed execution process

Cache Reused Data

Persistence reuses computed data; lineage records how missing data can be rebuilt



- Keep an intermediate RDD when later actions reuse it
- Reuse available cached partitions
- Recompute missing partitions from their lineage

Reuse One Intermediate RDD

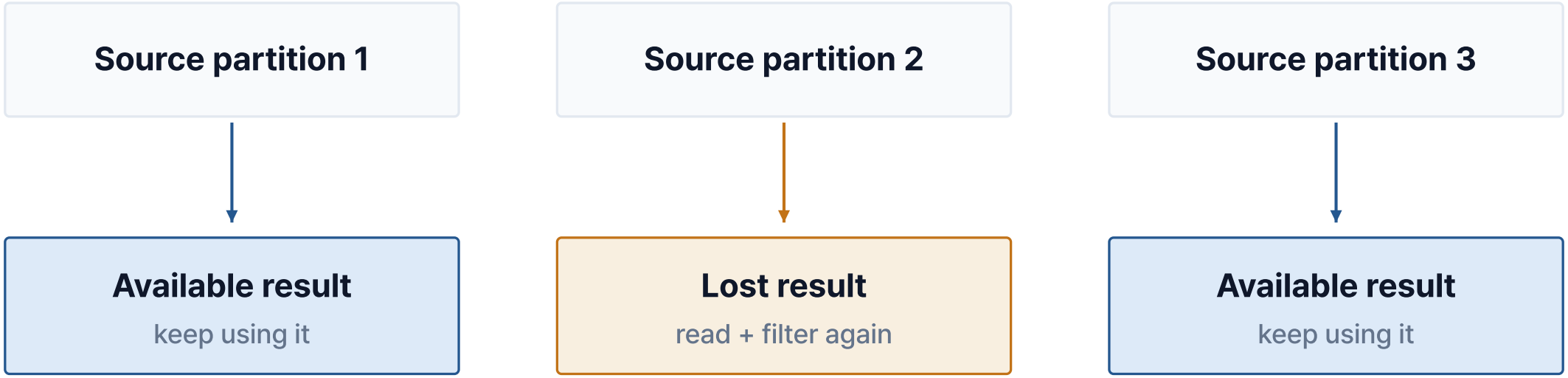
Caching lets later actions reuse available computed partitions

```
errors = sc.textFile("logs/").filter(
    lambda line: line.startswith("ERROR")).cache()

errors.count()  # Compute and keep the filtered partitions
errors.filter(lambda line: "sqlite" in line).count()
errors.unpersist()  # Release cached data when finished
```

Recover Missing Work

Lineage is the recorded path from input to result, used to recompute lost work



Lineage = dependencies and operations used to rebuild missing data

Independent filters rebuild the lost partition; dependencies determine wider retries

One Join Query, Two Small Inputs

Keep R.B > 200 and S.C < 100, join matching A values, then count the joined rows

R.A	R.B
1	250
2	180
3	300

S.A	S.C
1	50
1	120
3	80

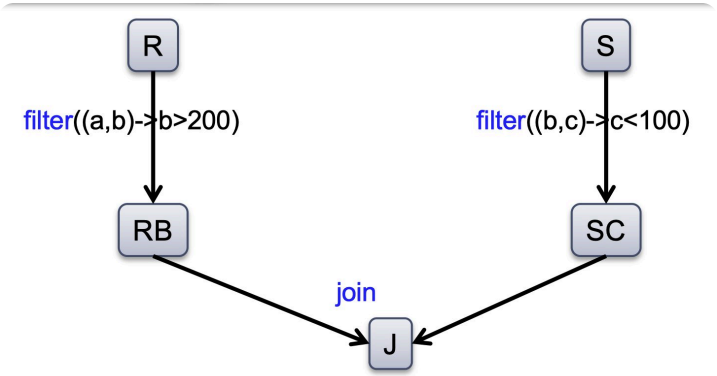
```
SELECT COUNT(*)
FROM R JOIN S ON R.A = S.A
WHERE R.B > 200 AND S.C < 100
```

Keep matching A values after filtering: (1,250,50) and (3,300,80). Count = 2.

A Join Query as RDD Operations

For R(A,B) and S(A,C), filter both inputs, join by A, then count

```
R = sc.parallelize([(1,250),
(2,180),(3,300)])
S = sc.parallelize([(1,50),(1,120),
(3,80)])
RB = R.filter(
    lambda row: row[1] > 200)
SC = S.filter(
    lambda row: row[1] < 100)
J = RB.join(SC)
answer = J.count()
```



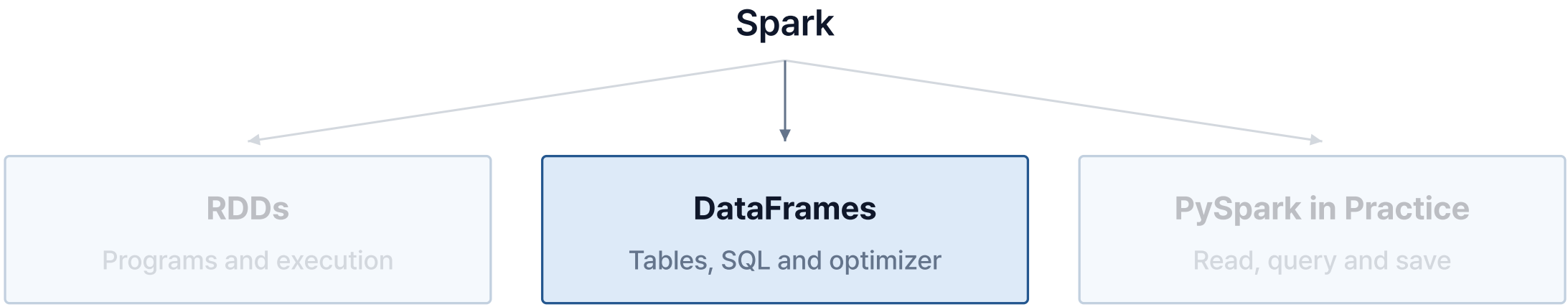
What RDDs Simplify

RDDs remove much of the job wiring, but the program still works with individual objects

MapReduce program	Spark RDD program
Write mapper and reducer classes	Call map, filter, join or reduceByKey
Wire a sequence of jobs and intermediate files	Compose operations in one program
Manage key/value logic for each operation	Reuse built-in distributed operations

We still write functions and choose the operation sequence

§ DataFrames


Spark
RDDs
Programs and execution
DataFrames
Tables, SQL and optimizer
PySpark in Practice
Read, query and save

Why DataFrames

Named columns let us express the query as table operations

```
R.filter(lambda row: row[1] > 200)
```

What is row[1]? The programmer must know the record layout.

- A DataFrame gives columns names and types
- Table operations describe the query in a form Spark can optimize

The Same Join with DataFrames

The same rows now have named columns; the filtered join still counts two matches

```
R = spark.createDataFrame([(1,250),(2,180),(3,300)], ["A","B"])
S = spark.createDataFrame([(1,50),(1,120),(3,80)], ["A","C"])

answer = (R.filter(R.B > 200)
    .join(S.filter(S.C < 100), on="A")
    .count())
```

The Same Join in Spark SQL

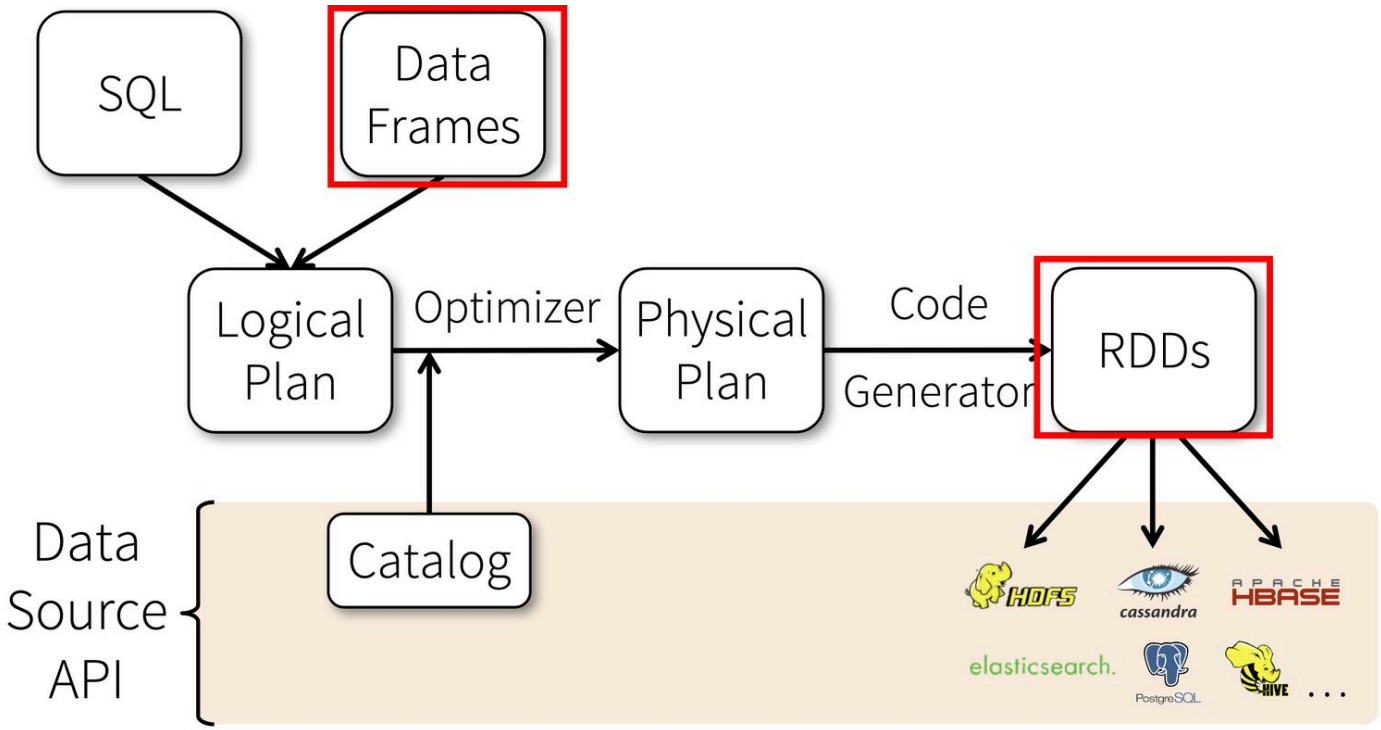
The familiar SQL query runs through Spark's distributed execution engine

```
R.createOrReplaceTempView("R")
S.createOrReplaceTempView("S")

answer = spark.sql("""
    SELECT COUNT(*) AS n
    FROM R JOIN S ON R.A = S.A
    WHERE R.B > 200 AND S.C < 100
""")
answer.show()
```

DataFrames and the Optimizer

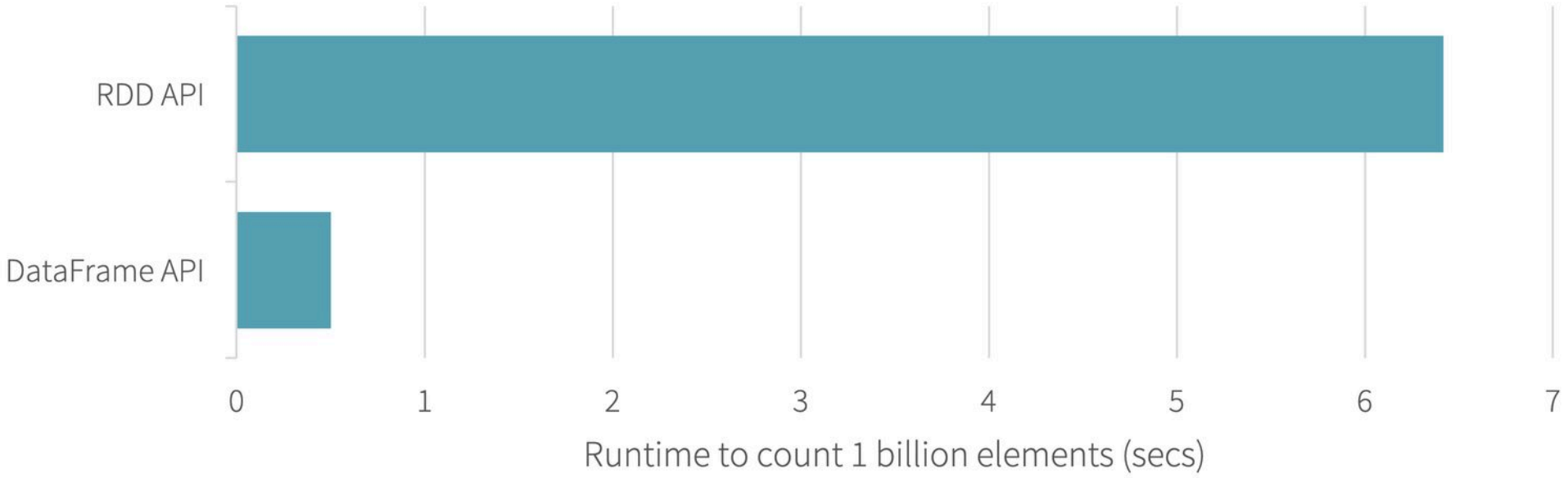
SQL and DataFrame expressions produce a plan the optimizer can inspect



Describe the table query; let Spark choose an execution plan

A Historical DataFrame Performance Example

This historical count benchmark illustrates how a table API can enable optimized execution



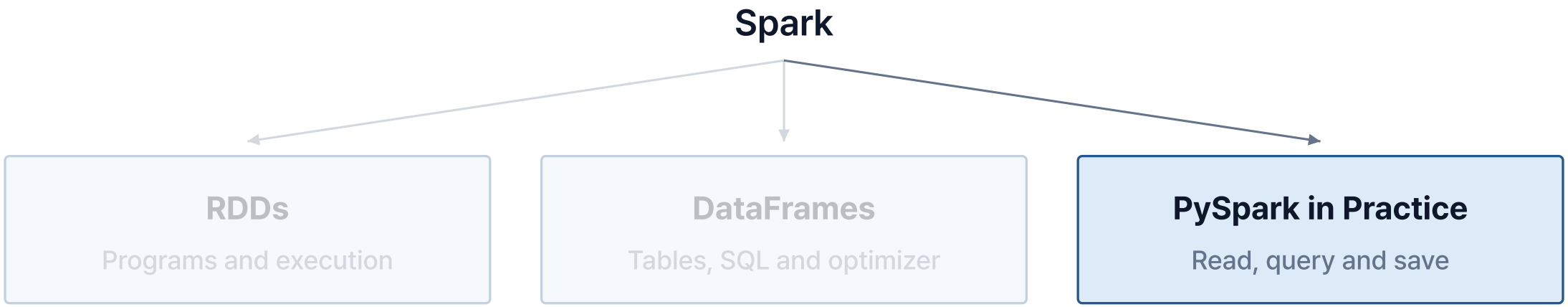
The pictured workload is one example; the speed difference depends on the query and environment

MapReduce, RDDs and DataFrames

Each step lets us express more of the task through a higher-level interface

	MapReduce	Spark RDD	Spark DataFrame / SQL
You write	map/reduce functions and job setup	Functions and operation chains	Columns, table operations or SQL
You manage	Keys, source tags and job boundaries	Object layout and operation sequence	Schema and query meaning
The engine provides	Distribution, shuffle and retries	Distributed operators and lineage	Those execution services plus SQL optimization

§ PySpark in Practice



From a CSV File to a Table

A comma-separated values (CSV) file stores one table row per line

readings.csv: values separated by commas

```
id,machine,temp
1,M1,70
2,M2,90
3,M1,85
```

events: the same values as table rows

id	machine	temp
1	M1	70
2	M2	90
3	M1	85

First line: column names (header)

Schema: column names + value types

The header names columns; the schema gives those columns their value types

Read Files with a Schema

A schema defines the columns: id is a whole number, machine is text, temp is numeric

```
events = (spark.read
    .schema("id INT, machine STRING, temp DOUBLE")
    .option("header", True)  # First CSV row contains column names
    .csv("raw/readings/2026-11-05/"))  # Folder for one day
```

One Sensor Reading

Each row records one temperature measurement from one machine

id	machine	temp
3	M1	85



Reading number

measurement 3

Machine name

M1 = one machine

Temperature

85 degrees Celsius

Row 3 says that machine M1 was measured at 85°C

Filtering Hot Readings

For one day of readings, keep every measurement at 80°C or above

Nine measurements from one day

id	machine	temp
1	M1	70
2	M2	90
3	M1	85
4	M1	95
5	M3	75
6	M2	60
7	M3	88
8	M1	82
9	M2	78



Five measurements at 80°C or above

id	machine	temp
2	M2	90
3	M1	85
4	M1	95
7	M3	88
8	M1	82

Keep a row if temp >= 80

The condition keeps five of the nine readings

Counting by Machine

Put the same machine name together, then count the readings in each group

M1 readings

id	temp
3	85
4	95
8	82

3 hot readings

M2 readings

id	temp
2	90

1 hot reading

M3 readings

id	temp
7	88

1 hot reading

Group by machine = put the same machine name together

M1 has three hot readings; M2 and M3 each have one

Filtering a DataFrame

filter keeps matching rows; hot names that DataFrame; show() requests its rows

```
hot = events.filter(events.temp >= 80)
hot.show()  # Request the rows
```

id	machine	temp
2	M2	90
3	M1	85
4	M1	95
7	M3	88
8	M1	82

Result: five individual readings

The selected rows keep their original id, machine and temperature columns

Counting Groups in a DataFrame

groupBy chooses the grouping column; count defines one row count for each group

```
# Group rows by machine, then count each group
hot_counts = hot.groupBy("machine").count()
hot_counts.show()  # Request the grouped rows
```

machine	count
M1	3
M2	1
M3	1

Result: one count per machine

hot_counts describes three grouped rows rather than five individual readings

The Same Question in SQL

WHERE keeps the hot rows; GROUP BY machine and COUNT(*) count them per machine

```
events.createOrReplaceTempView("events")  # Name the table for SQL

hot_counts = spark.sql("""
    SELECT machine, COUNT(*) AS count
    FROM events
    WHERE temp >= 80
    GROUP BY machine
""")
hot_counts.show()  # Request the same three grouped rows
```

Two Meanings of count()

The object before count() determines whether it returns a number or defines grouped rows

Count all hot readings

hot.count()

action: returns one Python integer

5

Count within each machine

hot.groupBy("machine").count()

transformation: defines a DataFrame

machine	count
M1	3
M2	1
M3	1

Call .show() to request these rows

Five hot readings and three grouped result rows answer different questions

Demo: Change the Query Plan

Browser simulation: change the condition, predict the rows, then run an action

Keep temp >= 80 | Group by machine | Run action | Reset

Input P1 id / machine / temp	Input P2 id / machine / temp	Input P3 id / machine / temp
1 / M1 / 70	4 / M1 / 95	7 / M3 / 88
2 / M2 / 90	5 / M3 / 75	8 / M1 / 82
3 / M1 / 85	6 / M2 / 60	9 / M2 / 78

Plan only

filter → group by machine → count

Condition changed: predict the result, then press Run action. This browser model is not Spark.

The grouping choice changes the meaning and number of output rows

Answer: Change the Query Plan

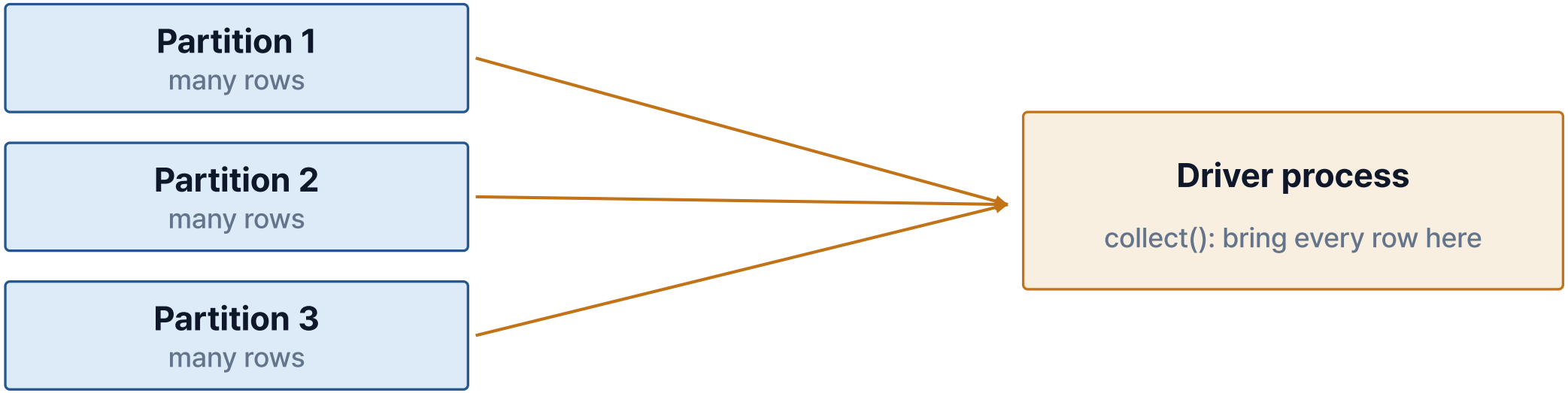
At 85, one M1 reading drops out; the other machines remain

Condition	With grouping	Without grouping
temp >= 80	M1=3, M2=1, M3=1	5 readings
temp >= 85	M1=2, M2=1, M3=1	4 readings
temp >= 100	0 rows	0 rows

Four retained readings produce three grouped rows

Caveat: Collecting Too Much

collect() gathers every result row into the driver; a large result may not fit



A distributed input can become a single-machine memory problem

Check the result size before turning distributed data into a local object

Write Distributed Results

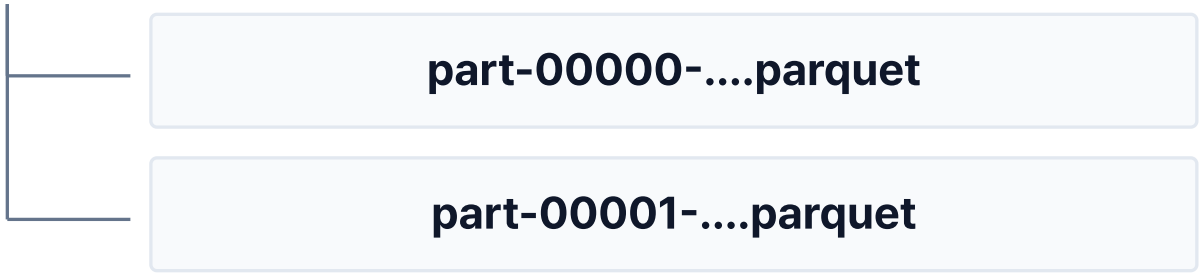
write saves the table; Parquet is a column-based file format for the saved data

```
(hot_counts.write
    .mode("errorifexists")  # Stop if the path exists
    .parquet("output/hot_counts/run_2026_11_05/"))
```

A Dataset Is Often a Folder

Different data pieces can be saved as separate files and read together as one table

hot_counts/run_2026_11_05/



Output is a folder

with data files

Read the folder as a table

Several part files can represent one logical result table

Caveat: Overwriting the Wrong Path

Writing one day of results to the whole history folder can replace other days



Writing one day to the whole path can remove other days

A one-day input does not make a whole-path overwrite safe

One Complete Spark Job

Define a bounded input, transform it, then save the result once

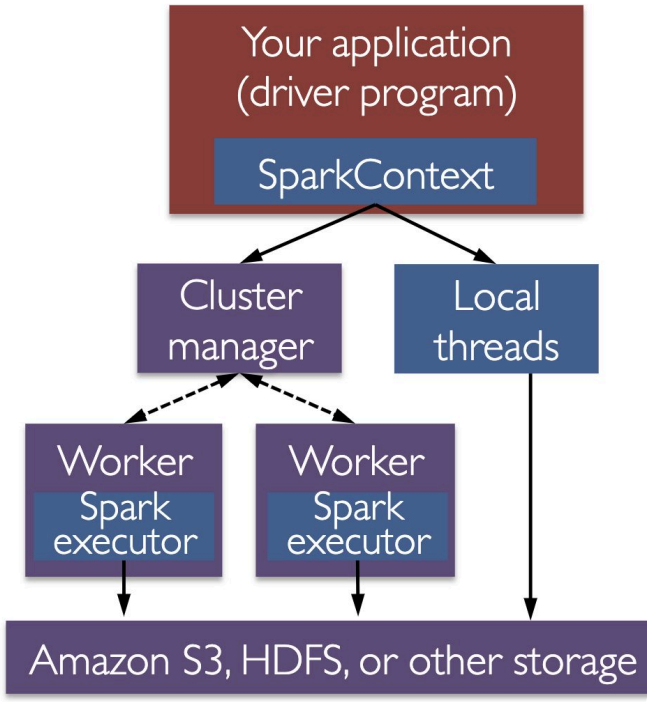
```
events = (spark.read.schema("id INT, machine STRING, temp DOUBLE")
    .option("header", True).csv("raw/readings/2026-11-05/"))

hot_counts = (events.filter(events.temp >= 80)
    .groupBy("machine").count())

(hot_counts.write.mode("errorifexists")
    .parquet("output/hot_counts/run_2026_11_05/"))
```

PySpark: Local Mode or a Cluster

The same programming interface can run on one computer or use executors across a cluster



The driver coordinates the work; execution resources can be local or distributed

PySpark: RDD Operations to Keep Nearby

These small collection examples explain the operations used in the word-count program

```
sc = spark.sparkContext
rdd = sc.parallelize([1, 2, 3, 4])
rdd.map(lambda x: x * 2).collect()       # [2, 4, 6, 8]
rdd.filter(lambda x: x % 2 == 0).collect() # [2, 4]
rdd.distinct().collect()  # Unique values, order not guaranteed
sc.parallelize(["a b", "b c"]).flatMap(lambda x: x.split()).collect()
# ["a", "b", "b", "c"]
```

- map: one output object per input object
- flatMap: flatten each input's output sequence
- collect: bring all result objects to the driver, only for small results

PySpark: Group and Reduce by Key

The grouping key is the same; retaining all values and combining them have different costs

```
pairs = sc.parallelize([("a", 1), ("b", 4), ("a", 3)])

pairs.sortByKey().keys().collect()  # ["a", "a", "b"]

sorted(pairs.reduceByKey(lambda a, b: a + b).collect())
# [("a", 4), ("b", 4)]

sorted(pairs.groupByKey().mapValues(list).collect())
# [("a", [1, 3]), ("b", [4])]  # Value order is not guaranteed
```

For sums or counts, combine values with reduceByKey instead of keeping every value

PySpark: Small Results and Table Joins

Use the high-level table API, and keep the boundary between distributed and local data explicit

```
rdd.take(2)       # A small list of objects
rdd.reduce(lambda a, b: a + b)  # One combined value

# Only when the whole result fits in driver memory
pandas_df = result.toPandas()
spark_df = spark.createDataFrame(pandas_df)

# Join structured tables using a named key
joined = left.join(right, on="id", how="left")
```

- take returns a limited sample; collect and toPandas gather the whole result
- DataFrame joins support familiar SQL join types

PySpark: A Pair RDD Join

An inner join returns every matching pair within each key

```
x = sc.parallelize([("a", 1), ("b", 4)])
y = sc.parallelize([("a", 2), ("a", 3)])
sorted(x.join(y).collect())
# [("a", (1, 2)), ("a", (1, 3))]
```

- The first element is the join key
- One a on the left matches both a values on the right
- Key b has no partner, so this inner join emits no b row

Takeaway: MapReduce, RDDs and DataFrames

1. **MapReduce**: write map/reduce functions, keys and job setup
2. **Spark RDDs**: compose distributed operations in a shorter program
3. **DataFrames and SQL**: describe named-column queries and use the optimizer